

数字电路与逻辑设计

第 1 章 数字逻辑基础

张江山

zhangjs@hust.edu.cn

信息工程系

● 课程性质

- ◆ 是电子信息类专业重要的专业基础课程，核心课程
- ◆ 48 / 56 学时， 3 / 3.5 学分
- ◆ 是后续专业课程的基础：
 - 计算机原理与接口技术、 DSP 原理及应用、嵌入式系统等

● 课程目的及要求：

- ◆ 理解数字电子相关基本理论
- ◆ 掌握基本知识和基本技能
- ◆ 培养分析和解决问题的能力

- 课程成绩

- ◆ 平时成绩： 20% ~ 30%
- ◆ 期末考试成绩： 80% ~ 70%

- MOOC 课程

- ◆ 网址： <http://www.icourse163.org/course/HUST-1001909001>
- ◆ 注册： 昵称务必为“班号 + 姓名”，如：“电子 1601 张力”

- 签到： 使用微助教

- ◆ 微信中关注“微助教”公众号
- ◆ 注册： 确保学号、姓名无误

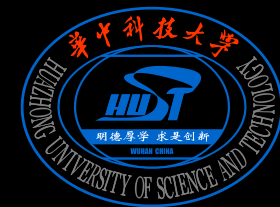
● 教材

- ◆ 康华光《电子技术基础 - 数字部分》，高等教育出版社

◆ 参考资料

- ◆ 罗杰《数字电子技术》，高等教育出版社
- ◆ 罗杰《电子技术基础数字部分习题全解》，高等教育出版社
- ◆ **IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005**
- ◆ **IEEE Standard Graphic Symbols for Logic Functions, ANSI/IEEE Std 91a-1991**

1. 数字逻辑概论：基本概念
2. 逻辑代数与硬件描述语言概述：基本理论
3. 逻辑门电路：由开关器件组成的逻辑门电路
4. 组合逻辑电路：组合逻辑电路的分析和设计方法
5. 锁存器与触发器：时序逻辑电路的基本单元
6. 时序逻辑电路：时序逻辑电路的分析和设计方法
7. 存储器： **ROM** 和 **RAM**
8. 复杂可编程器件和现场可编程门阵列： **CPLD** 和 **FPGA**
9. 脉冲波形变换与产生：单稳态、施密特触发器和多谐振荡器
10. 模数与数模转换器： **ADC** 和 **DAC** 的基本原理



- 1.1 数字电路与数字信号：基本概念
- 1.2 数制：数的表示方法
- 1.3 二进制数的算术运算：数的运算方法
- 1.4 二进制代码：码制，事物的二进制命名规则
- 1.5 二值逻辑变量与基本逻辑运算：逻辑运算方法
- 1.6 逻辑函数及其表示方法：逻辑函数式

- 了解数字逻辑电路的历史、分类及数字信号的描述方式
- 理解数制和码制基本概念
- 掌握带符号负数的二进制原码、反码、补码三种表示形式
- 掌握二进制算术加减运算方法
- 掌握 8421 码、余 3 码、格雷码的表示方法
- 掌握与、或、非、与非、或非、异或、同或等逻辑运算
- 掌握逻辑函数及其表示方法

- 电子技术， 20 世纪发展最迅速的技术
- 数字电子技术， 广泛用于广播、电视、通信、医疗、控制、测量、交通、航空、军事、探测、文娱、家用电器等领域



数字电视



计算机



计算机网络



数码摄像机

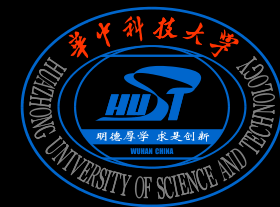


智能仪器

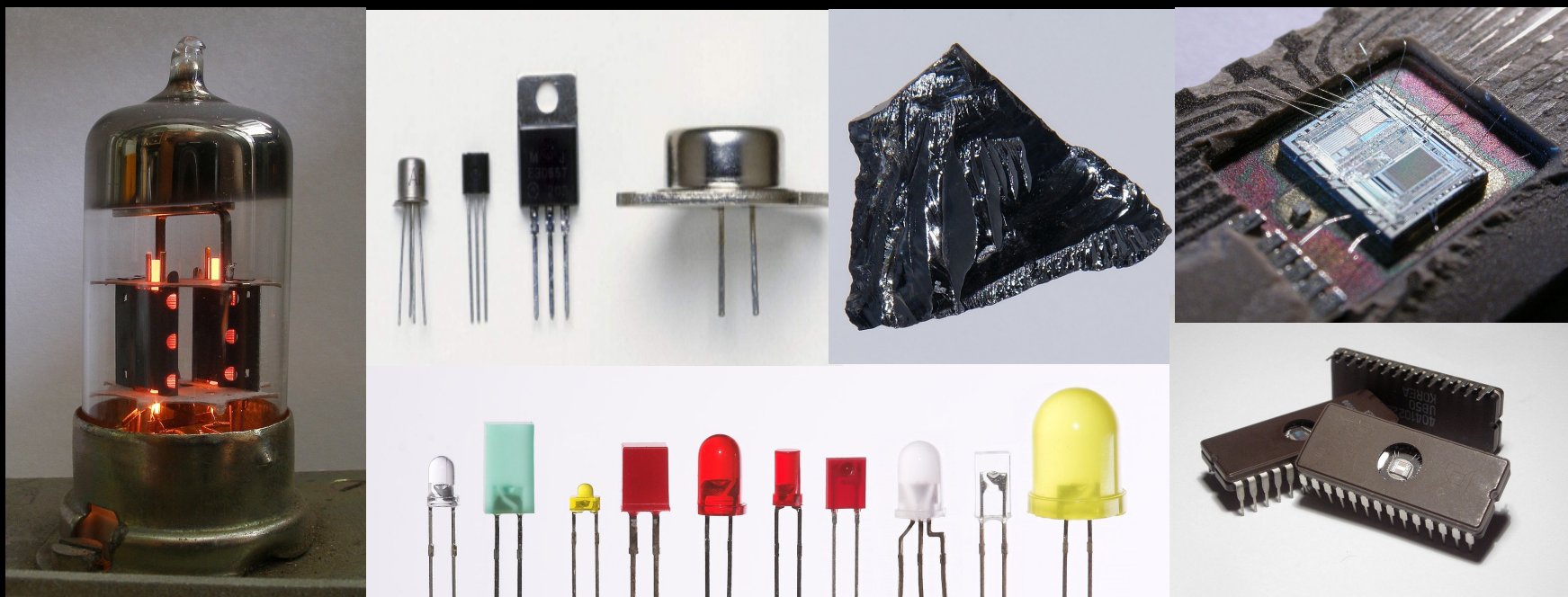


手机

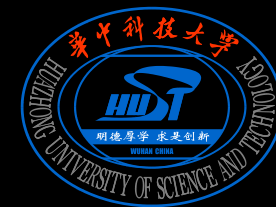
1.1.1 数字技术的发展及其应用



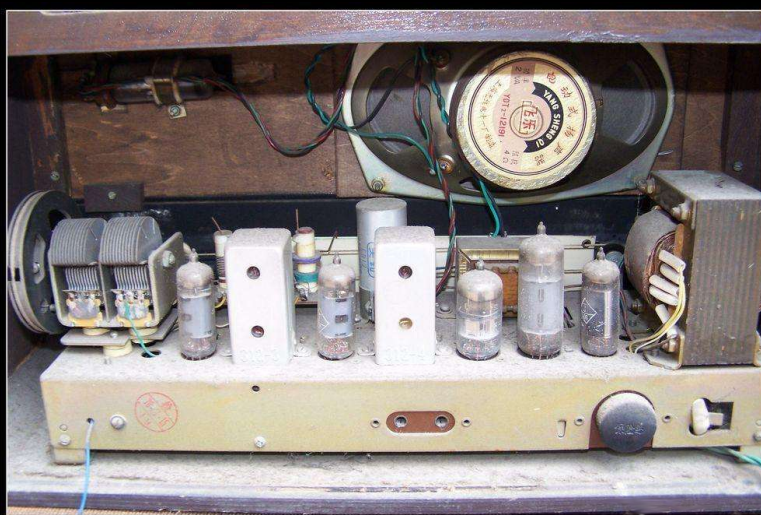
- 电子技术发展特点：以电子器件的发展为基础
- 电子管：1906 年，福雷斯特等发明了电子管，电真空技术
- 晶体管：1947 年第一只晶体三极管问世，半导体技术
- 半导体集成电路：20 世纪 60 年代初，模拟和数字集成电路



1.1.1 数字技术的发展及其应用



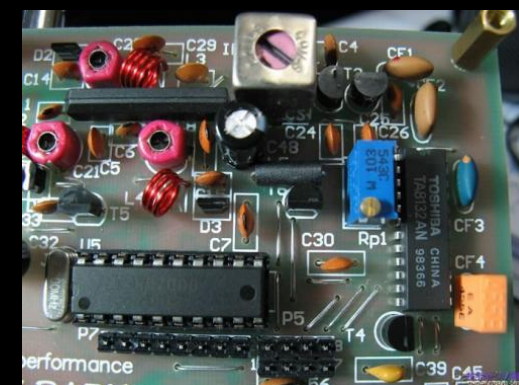
● 收音机



电子管

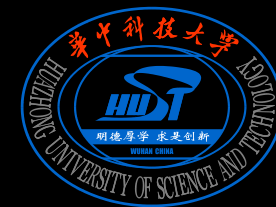


晶体管

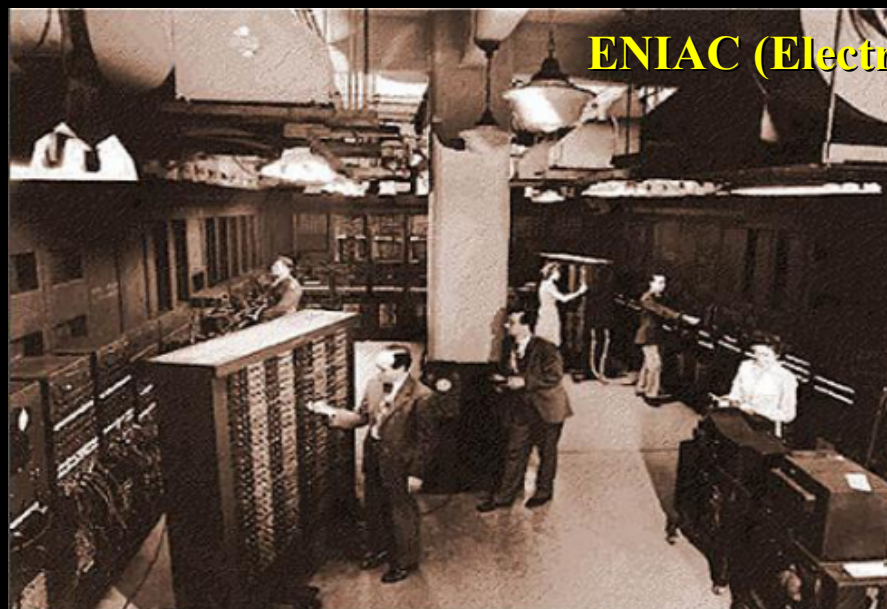


集成电路

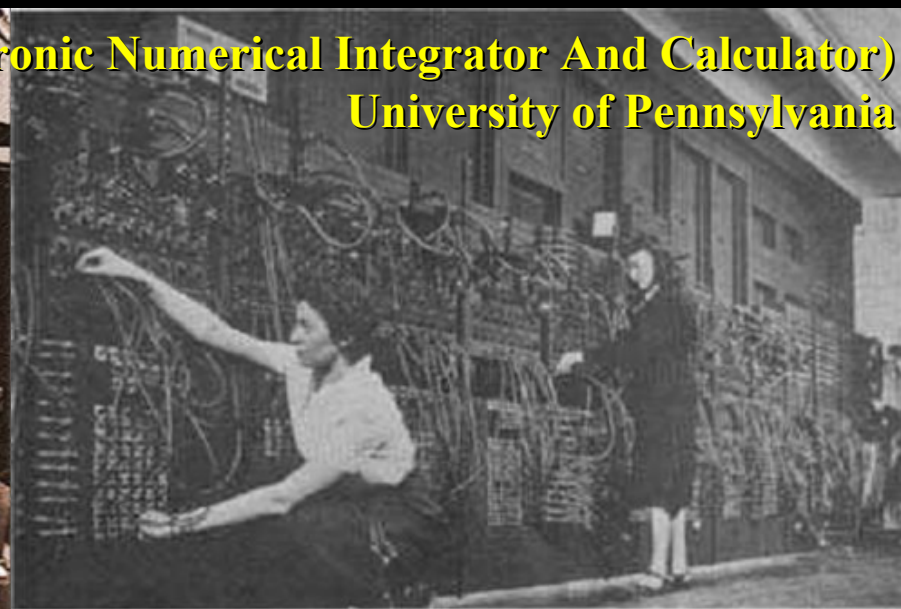
1.1.1 数字技术的发展及其应用



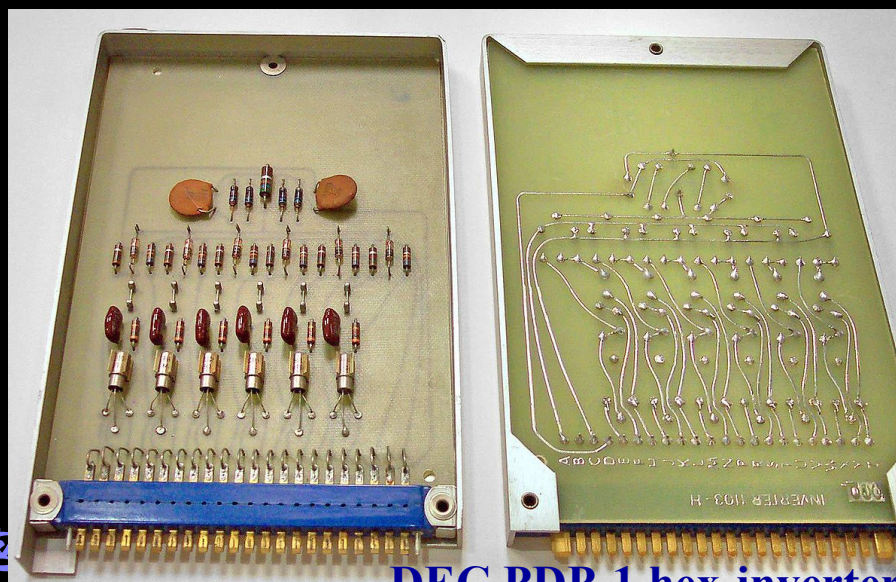
● 电子管计算机



ENIAC (Electronic Numerical Integrator And Calculator)
University of Pennsylvania



● 晶体管计算机



DEC PDP-1 hex-inverter

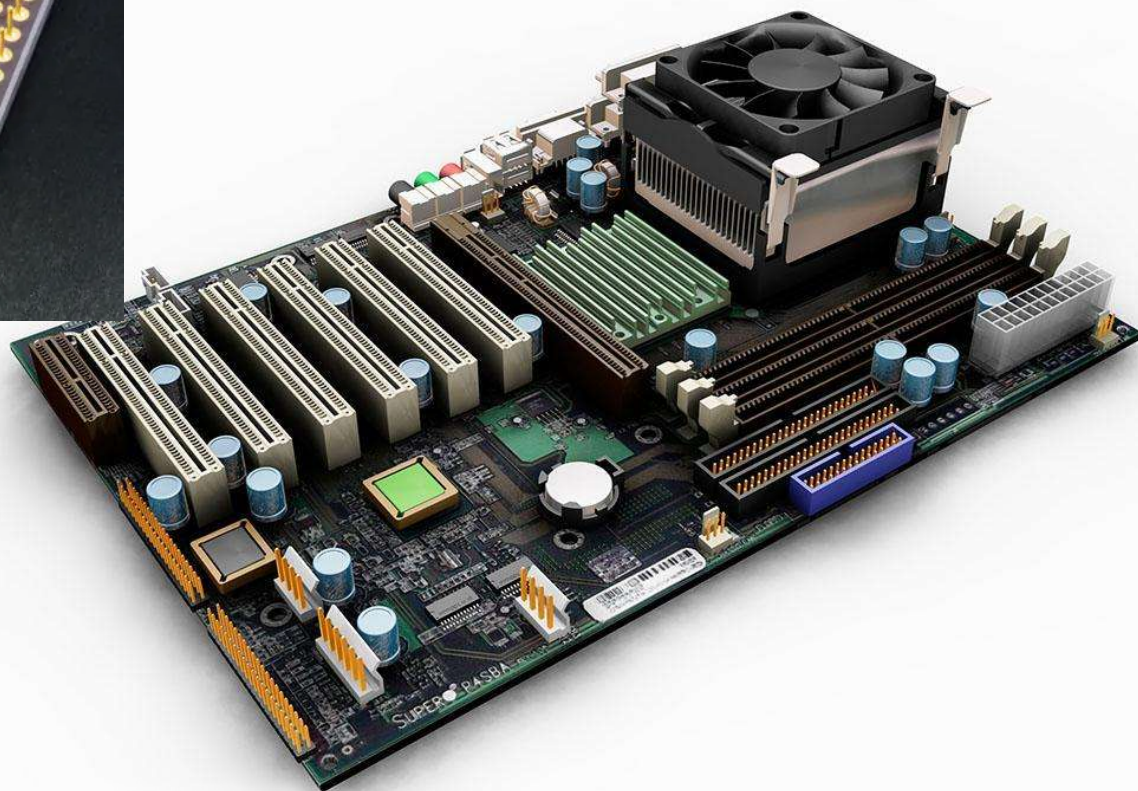
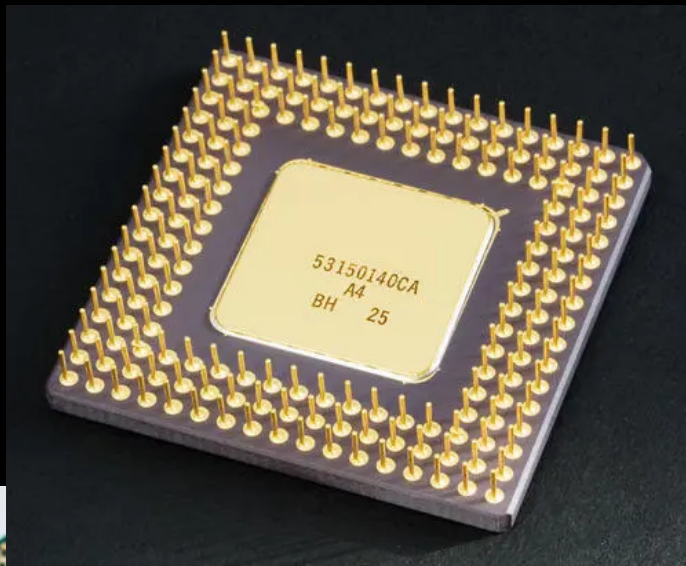
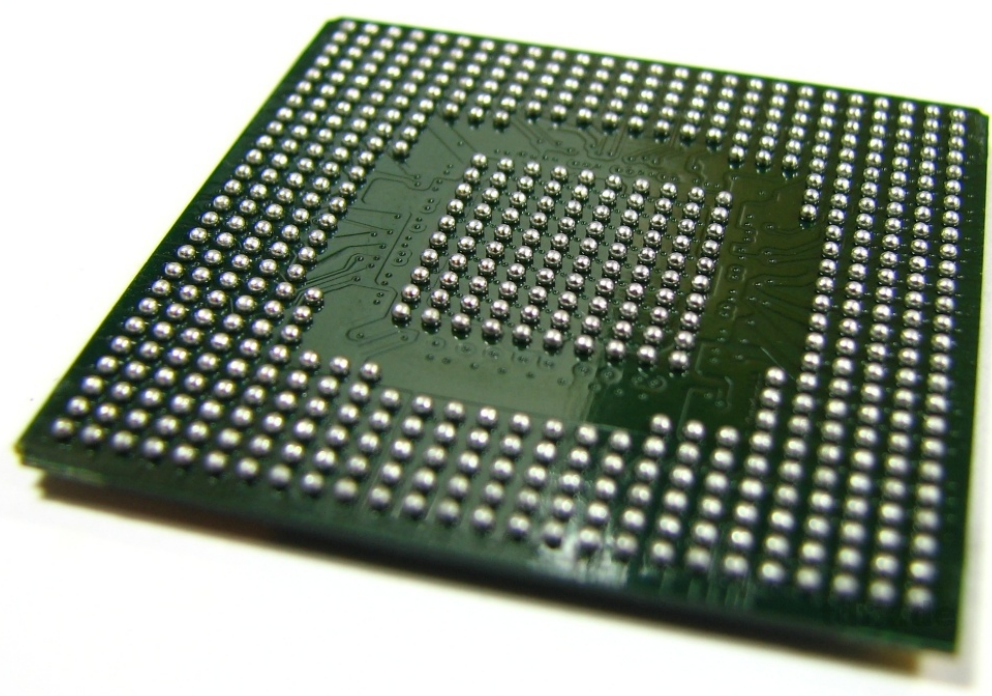


IBM 7094 console

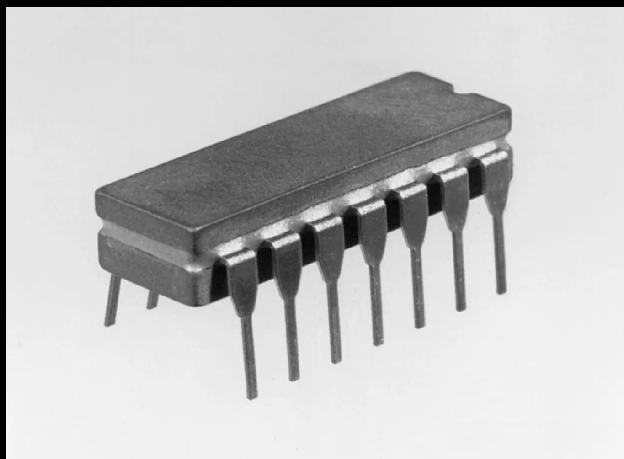
1.1.1 数字技术的发展及其应用

● 现代个人计算机 CPU 和主板

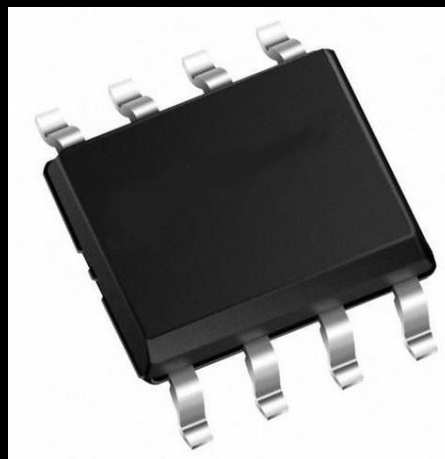
- PGA 插针阵列封装：体积较大，易拆卸
- LGA 平面阵列封装：体积适中，易拆卸
- BGA 焊球阵列封装：体积小，不易拆卸



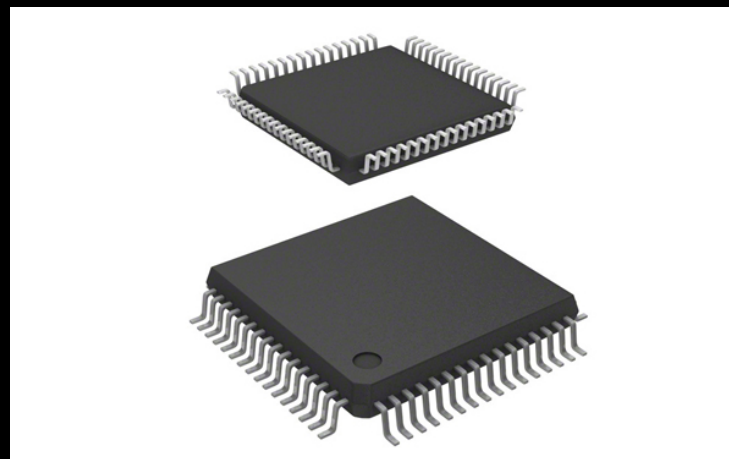
● 集成电路封装



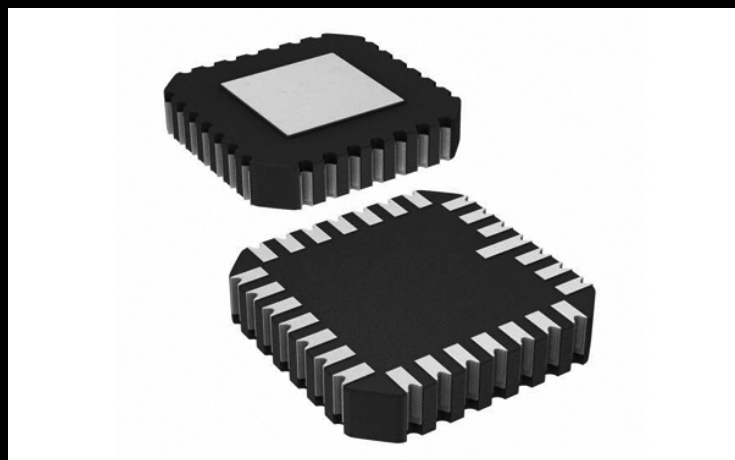
DIP 双列直插封装



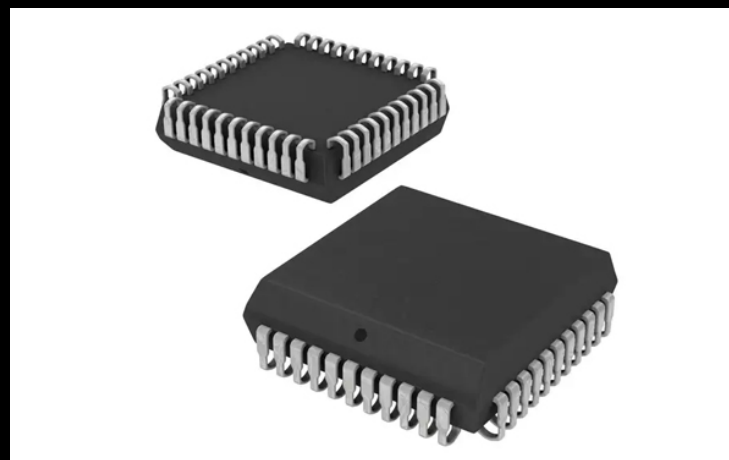
SOP 微型引脚封装



QFP 方形扁平封装



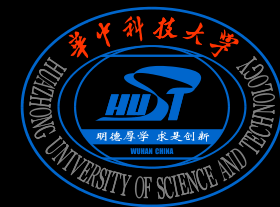
LCC 无引线封装



PLCC 有引线封装

1.1.1 数字技术的发展及其应用

- 电路设计方法，伴随着器件的发展，从传统走向现代
- 传统设计方法：
 - ◆ 逻辑抽象 → 逻辑代数 → 逻辑化简 → 逻辑电路 → 电路调试
- 现代设计方法：
 - ◆ EDA (Electronics Design Automation) 技术，设计软件化
 - ◆ 硬件描述 → 行为仿真 → 综合 → 时序仿真 → 网表下载
 - ◆ 将生成的网表数据下载到芯片，实现系统功能



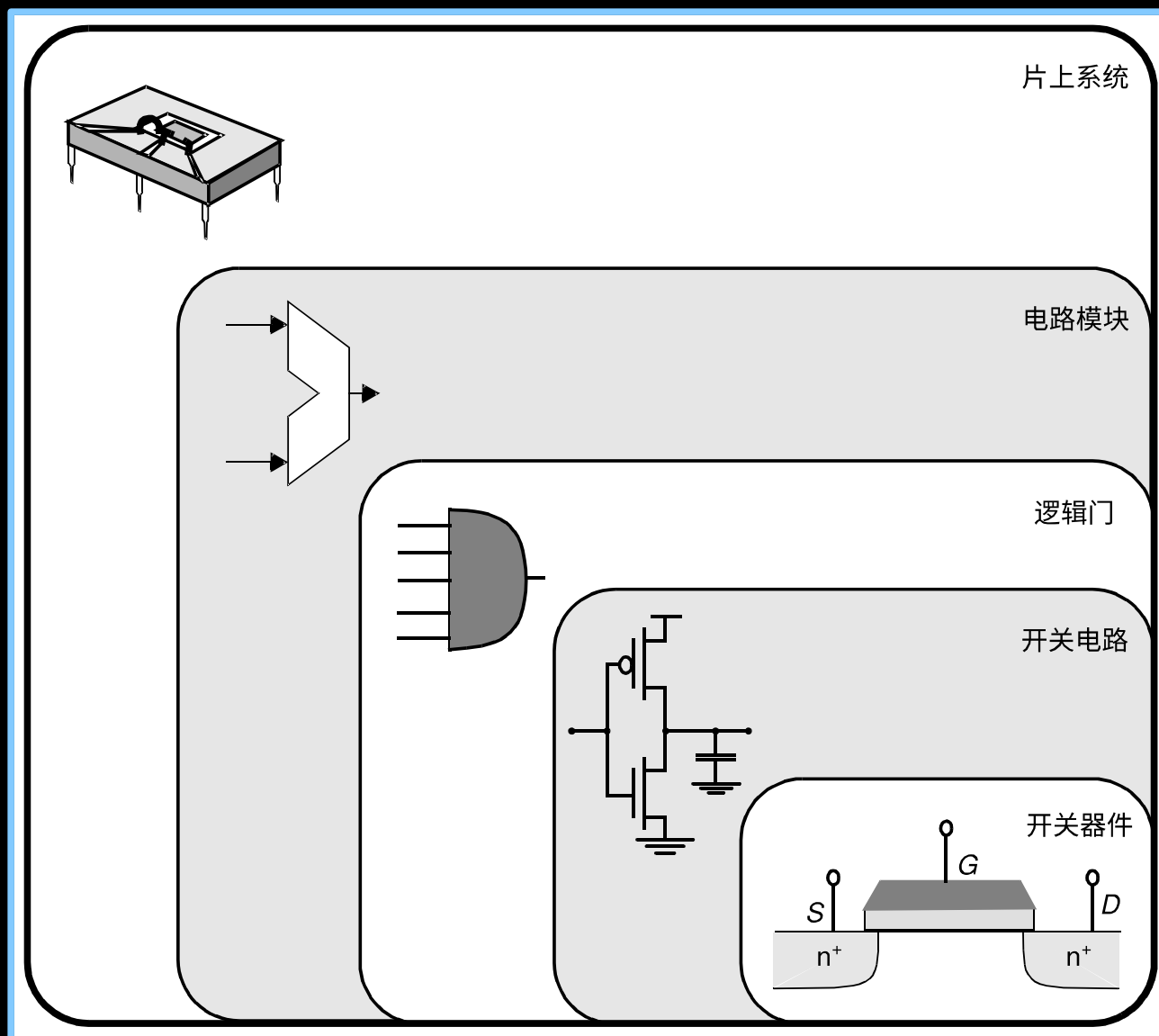
- 硬件描述语言 HDL
 - ◆ Verilog 和 VHDL 等
 - ◆ 可描述电路结构、数据流、行为及仿真过程
- 行为仿真工具： Icarus Verilog
 - ◆ 开源逻辑仿真工具， <https://github.com/steveicarus/iverilog>
 - ◆ iverilog（编译器）， vvp（仿真器）， gtkwave（示波器）
- FPGA 设计工具： Quartus (Altera) , Vivado (Xilinx)
- ASIC 设计工具： Cadence , Synopsys , Mentor

1.1.2 数字集成电路的分类及特点

- 电子电路分类：按功能分为**模拟**和**数字电路**
- 数字电路分类：按电路特点分为**组合**和**时序逻辑电路**
- 逻辑门电路：构成数字电路的基本单元
- 开关器件：组成逻辑门电路
 - ◆ 如二极管、三极管等都作为电子开关，或导通，或截止
- 数字集成电路：集成在半导体芯片上逻辑门电路
 - ◆ 集成度：芯片上所包含的逻辑门数量

1.1.2 数字集成电路的分类及特点

● 数字电路分级设计



ALU
FPU
MCU
MPU

加法器
比较器
选择器

与门
或门
非门

MOSFET
TTL
二极管
三极管

1.1.2 数字集成电路的分类及特点

- 数字集成电路的分类

- 按半导体器件分类

- ◆ 双极型

- TTL : 晶体管 - 晶体管逻辑电路

- ECL : 发射极耦合逻辑电路

- IIL : 集成注入逻辑电路

- ◆ MOS : 金属氧化物半导体

- PMOS : P 沟道

- NMOS : N 沟道

- CMOS : 混合沟道

- 按集成规模分类

- ◆ SSI : 小规模集成电路

- ◆ MSI : 中规模集成电路

- ◆ LSI : 大规模集成电路

- ◆ VLSI : 超大规模集成电路

- ◆ ULSI : 甚大规模集成电路

- 按设计方法和功能定义分类

- ◆ 非用户定制: 固定功能

- ◆ 半用户定制: 部分可编程

- ◆ 全用户定制: 完全可编程

- 模拟信号：表示模拟量的信号

- ◆ 模拟量：模拟自然界中随时间连续变化物理量的参考量

- 如温度、压力

- ◆ 模拟电路：处理模拟信号的电子电路

- 使用传感器将模拟量转换为成比例的电压或电流信号

- 数字信号：表示数字量的信号

- ◆ 数字量：用数字表示某些时刻离散变化物理量的参考量

- 如：每小时测一次温度，每次测量的最小单位为 1°C

- ◆ 数字电路：处理数字信号的电子电路

- ◆ 问题：数字信号相较于模拟信号的优点有哪些？

- A. 抗干扰强， B. 易加密， C. 便于电脑处理， D. 易传输， ~~E. 高保真~~

● 数字电路的分析、设计

◆ 数字电路分析

- 已知逻辑电路，确定输出、输入之间的逻辑关系
- 分析工具：逻辑代数
- 逻辑关系描述方式：真值表、功能表、逻辑表达式和波形图

◆ 数字电路设计

- 已知功能要求，选择器件，设计出符合要求的逻辑电路
- 设计工具
 - 传统：逻辑代数
 - 现代：EDA 硬件描述语言（结构建模，数据流建模，行为建模）

1.1.4 数字信号的描述方法

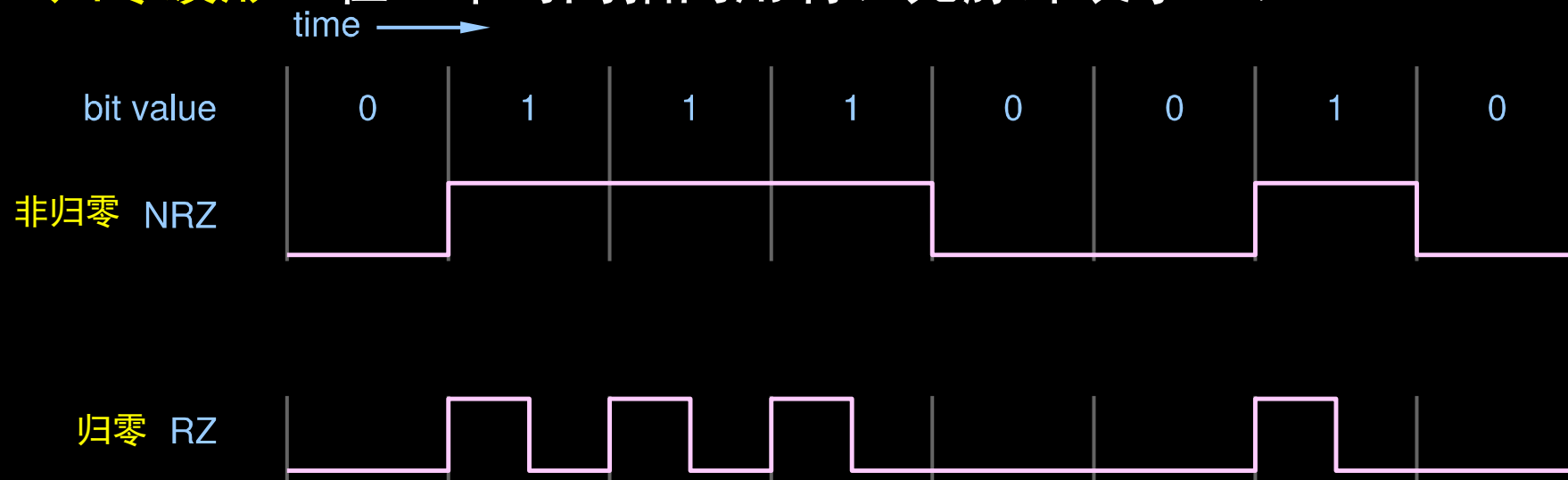
- 二值数
 - ◆ 用 0 和 1 两个符号描述的数值
- 二值逻辑
 - ◆ 用 0 和 1 两个符号描述的逻辑状态
- 逻辑电平
 - ◆ 正逻辑：用低 **L**、高 **H** 电平分别表示 **0** 和 **1**
 - ◆ 负逻辑：用低 **L**、高 **H** 电平分别表示 **1** 和 **0**

电压 (V)	正逻辑	电 平
+5	1	H(高电平)
0	0	L(低电平)

1.1.4 数字信号的描述方法

● 数字波形

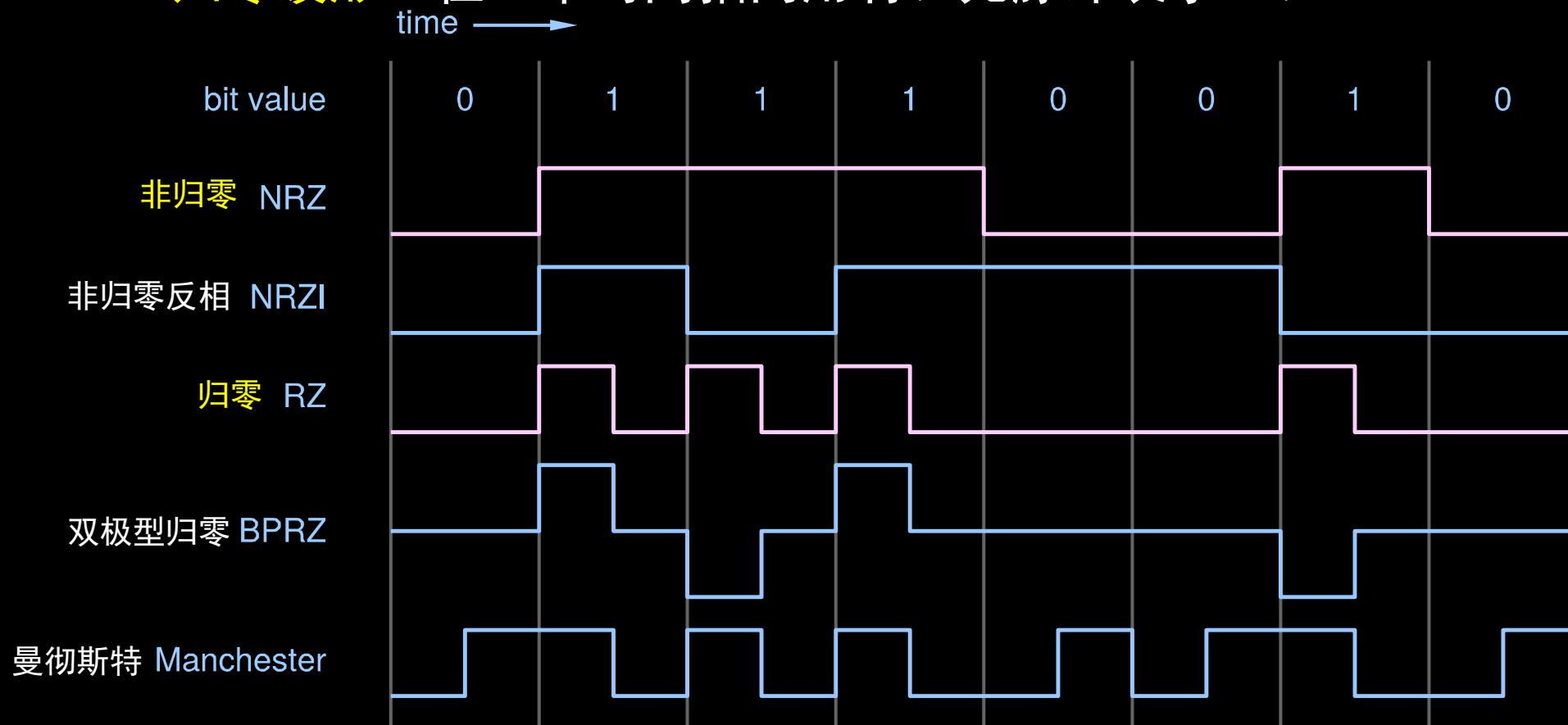
- ◆ 一个时间间隔（时间拍）的逻辑状态，称为 1 位 (bit)
- ◆ **NRZ 非归零波形**：在一个时间拍内用高、低电平表示 1、0
- ◆ **RZ 归零波形**：在一个时间拍内用有、无脉冲表示 1、0



1.1.4 数字信号的描述方法

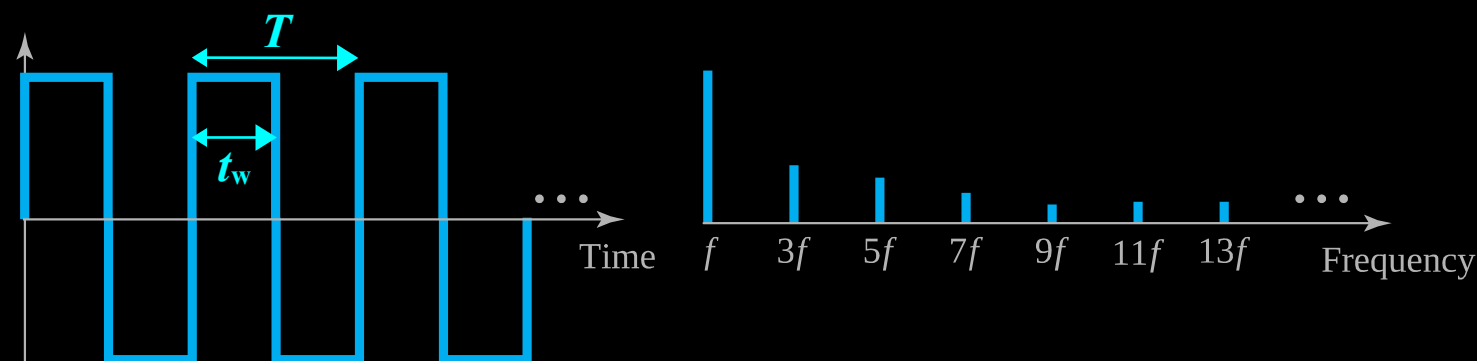
● 数字波形

- ◆ 一个时间间隔（时间拍）的逻辑状态，称为 1 位 (bit)
- ◆ **NRZ 非归零波形**：在一个时间拍内用高、低电平表示 1、0
- ◆ **RZ 归零波形**：在一个时间拍内用有、无脉冲表示 1、0

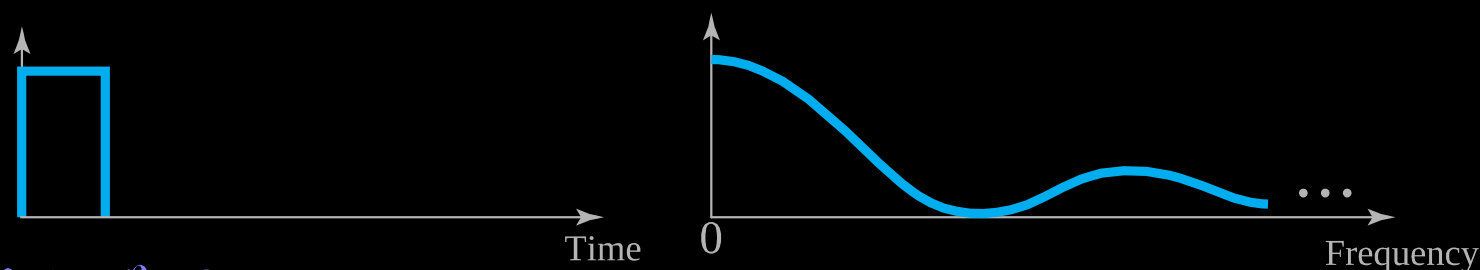


1.1.4 数字信号的描述方法

- 比特率：每秒钟传输数据的位数
- 周期性和非周期性信号
 - ◆ 脉冲宽度 t_w ：脉冲作用的时间
 - ◆ 周期 T 和频率 $f = \frac{1}{T}$
 - ◆ 占空比 $q = \frac{t_w}{T}$



a. Time and frequency domains of periodic digital signal



b. Time and frequency domains of nonperiodic digital signal

1.1.4 数字信号的描述方法

- 例：某系统数据传输率为 1.544 Mbps，求每位占用时间

- ◆ 解：按题意，每位占用时间为

$$(1.544 \times 10^6)^{-1} = 0.64767 \times 10^{-6} s = 648 ns$$

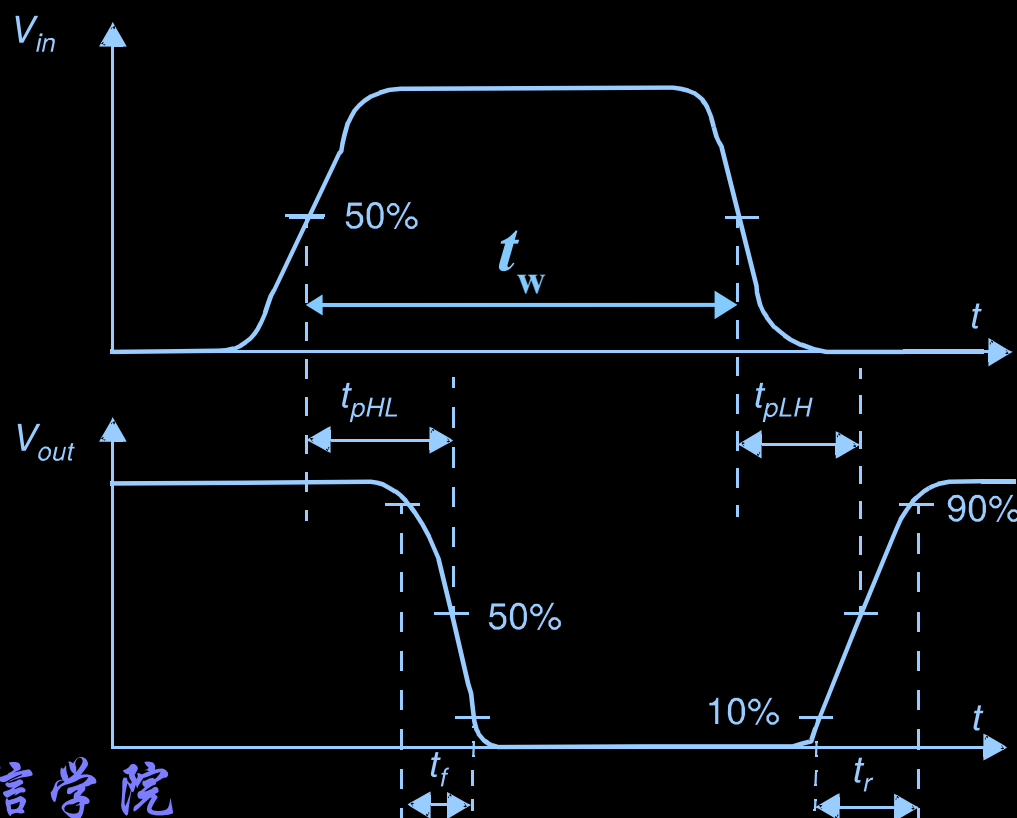
- 例：周期数字信号，高电平持续 6 ms，低电平持续 10 ms，求占空比 q

- ◆ 解：因脉冲宽度 $t_w = 6 ms$ ，周期 $T = 6 ms + 10 ms = 16 ms$

$$q = \frac{t_w}{T} \times 100\% = \frac{6 ms}{16 ms} \times 100\% = 37.5\%$$

● 实际数字信号波形

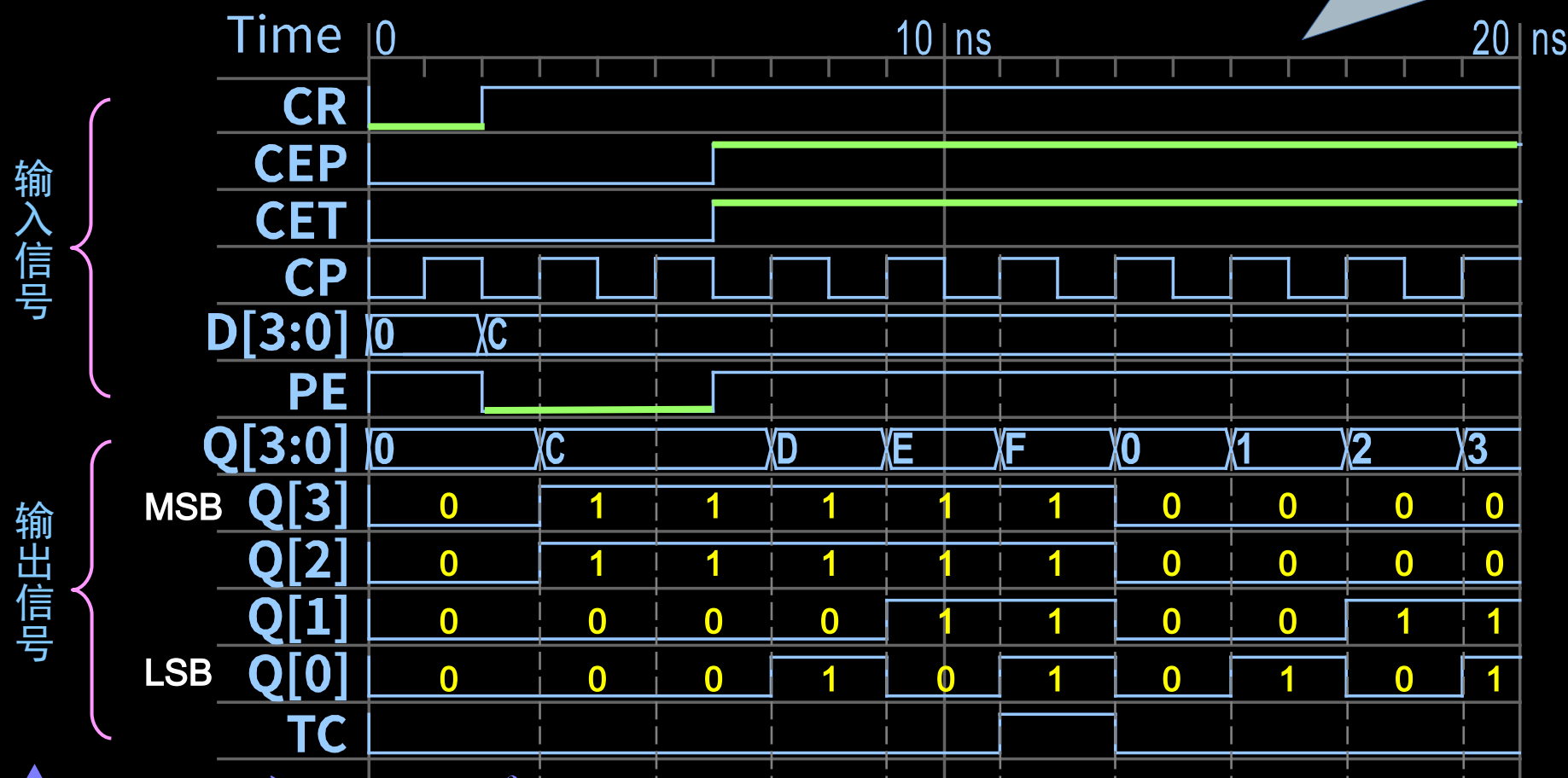
- ◆ 脉冲上升时间 t_r 和下降时间 t_f ：脉冲上升沿和下降沿的持续时间
- ◆ 脉冲宽度 t_w ：脉冲上升沿至下降沿 50% 幅值之间的持续时间
- ◆ 输出高变低电平延迟 t_{pHL} 和低变高电平延迟 t_{pLH} ：输出信号相对于输入信号变化的延迟时间



1.1.4 数字信号的描述方法

- 时序图：描述多个信号一起随时间顺序变化的多重波形图
 - ◆ 常用来分析逻辑电路的工作过程
 - ◆ 如图，CP 脉冲信号是数字系统的参考时钟

74HC161 计数器时序图



- 数（数量） / 码（代码）：数量大小 / 事物标识符
- 数制 / 码制：数值表示与进位计数规则 / 代码的表示规则
- 用特定的**进位基数**和**数位权值**表示数和进位计数规则
- **进位基数**：每位可使用数码符号的个数，简称**位模**，记作 R
 - ◆ 十进制， $R = 10$ ，符号为 $0 \sim 9$
 - ◆ 二进制， $R = 2$ ，符号为 $0、1$

问题：八进制， $R = ?$ ，符号为？

问题：十六进制， $R = ?$ ，符号为？
- **数位权值**：某位为 1 时，其表征的值，简称**位权**，记作 R^k
 - ◆ 其中 k 为位号，第 0 位是最低位， R 为位模
 - ◆ MSB (Most Significant Bit)：最高有效位
 - ◆ LSB (Least Significant Bit)：最低有效位，一般是第 0 位

问题：十进制的个位、十位、百位的位权分别是多少？

1.2.1 十进制

- 十进制数用下标 **D** 或 **10** 表示，也可省略
- 位模为 **10**，数字符号 **0~9**
- 逢十进一，比如： $1 + 9 = 10$
- 第 k 位的位权为 10^k
- 任意十进制数，都可按其位权展成多项式的形式：

$$(368.258)_D = 3 \times 10^2 + 6 \times 10^1 + 8 \times 10^0 + 2 \times 10^{-1} + 5 \times 10^{-2} + 8 \times 10^{-3}$$

- 任意十进制数 $(d_{N-1} \dots d_1 d_0 . d_{-1} d_{-2} \dots d_{-M})_D$ 可表示为： $x_D = \sum_{k=-M}^{N-1} d_k 10^k$

- 任意 R 进制数 $(r_{N-1} \dots r_1 r_0 . r_{-1} r_{-2} \dots r_{-M})_R$ 可表示为： $x_D = \sum_{k=-M}^{N-1} r_k R^k$

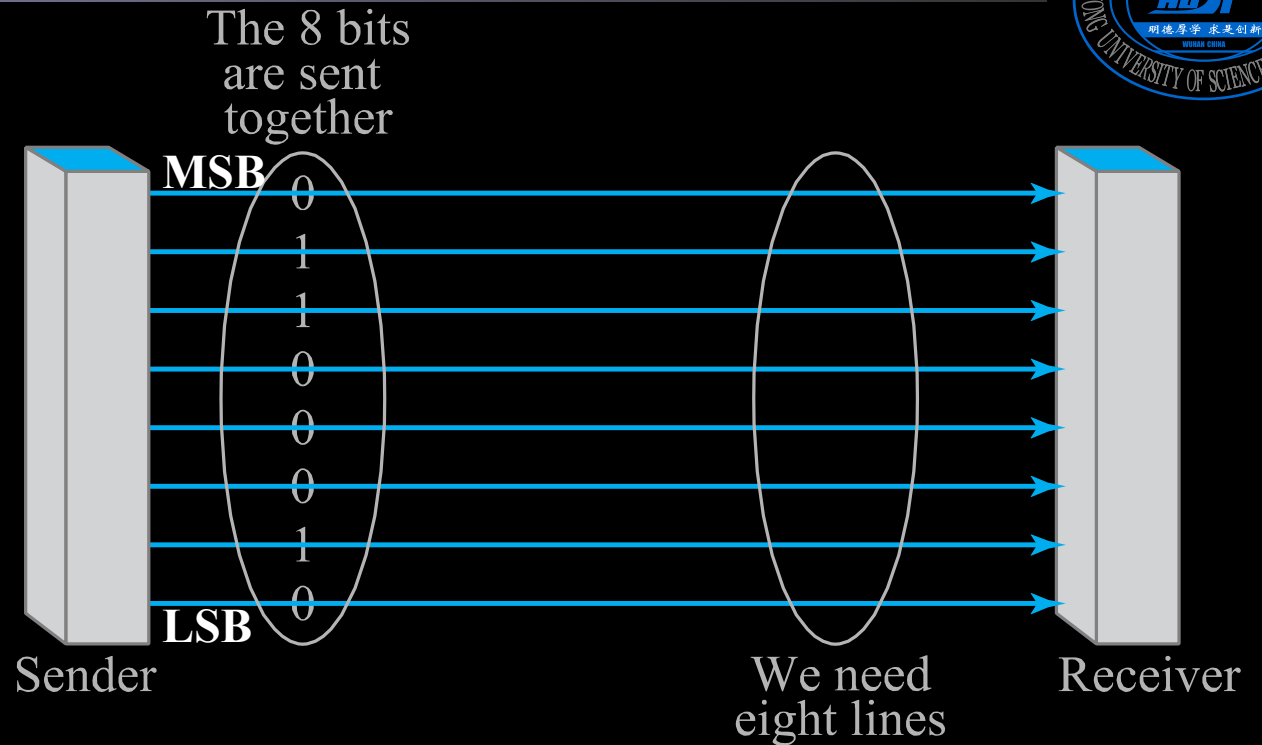
- 二进制数用下标 **B** 或 **2** 表示
- 位模为 2，数字符号 0 和 1
- 逢二进一，即 $1_B + 1_B = 10_B$
- 第 k 位的位权为 2^k
- 二进制数 $(b_{N-1} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-M})_B$ 位权多项式：
$$x_D = \sum_{k=-M}^{N-1} b_k 2^k$$
 - ◆ 即二进制数转十进制数方法：位权多项式按十进制求和

例 1.2.1 : $(10110.101)_B = (?)_D$

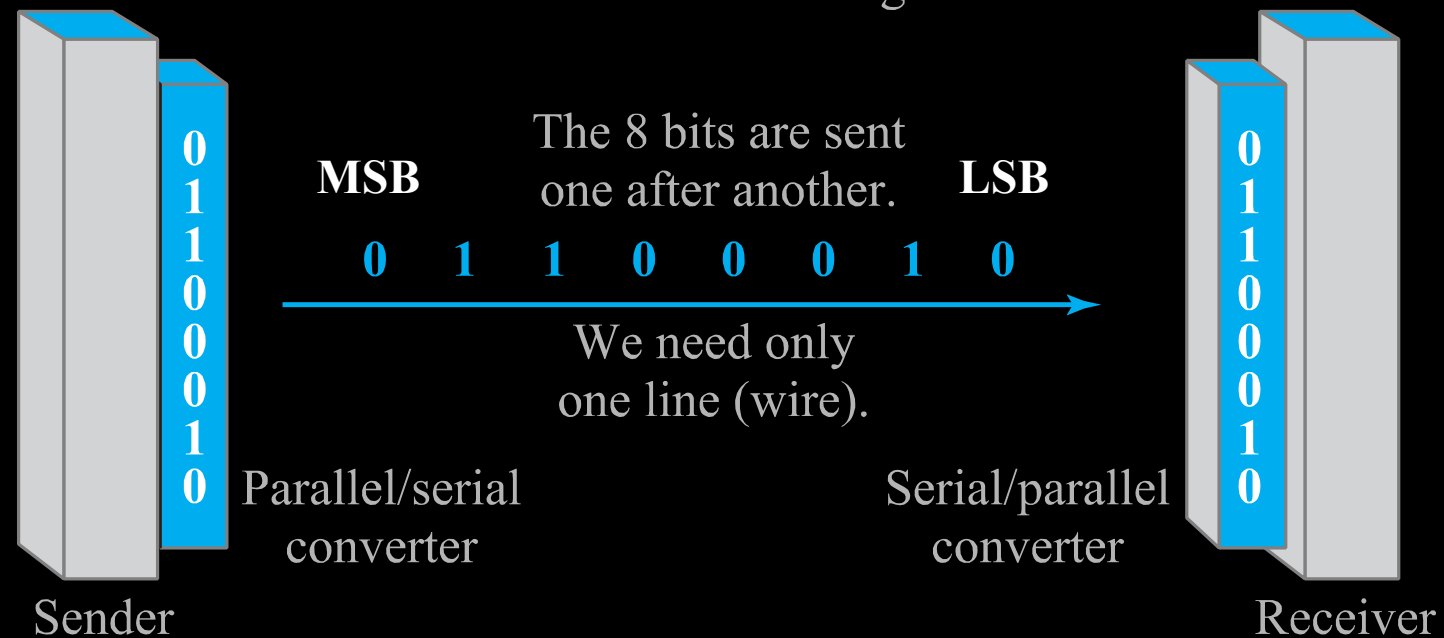
$$\begin{aligned}\text{解: } (10110.101)_B &= 2^4 + 2^2 + 2^1 + 2^{-1} + 2^{-3} \\ &= 16 + 4 + 2 + 0.5 + 0.125 \\ &= (22.625)_D\end{aligned}$$

1.2.2 二进制

● 并行传输方式



● 串行传输方式



1.2.3 二进制与十进制之间的转换

- 二进制转十进制数：对位权展开多项式，按十进制求和
- 十进制转二进制数：对整数和小数部分，用不同的方法转换
 - ◆ 整数部分采用短除法或凑数法转换
 - 整数短除法：除 2 取余，先低后高，商为 0 止
即将十进制数连续除以 2，直至商为零，所得余数由低到高位排列
 - 整数凑数法：按位权多项式由高至低逐位确定位权因子
 - ◆ 小数部分采用连乘法转换
 - 小数连乘法：乘 2 取整，先高后低，达标为止
即将十进制小数部分乘以 2，所得整数由高到低位排列

1.2.3 二进制与十进制之间的转换

- 短除法整数部分转换：除 2 取余，先低后高，商为 0 止
 - ◆ 将十进制数连续除以 2，直至商为零，所得余数由低到高位排列

例 1.2.2 : $(37)_D = (?)_B$

$$\begin{array}{r} 2 \overline{) 37} \\ 2 \overline{) 18} \dots\dots 1 \\ 2 \overline{) 9} \dots\dots 0 \\ 2 \overline{) 4} \dots\dots 1 \\ 2 \overline{) 2} \dots\dots 0 \\ 2 \overline{) 1} \dots\dots 0 \\ 0 \dots\dots 1 \end{array}$$

- 问题： $(1048577)_D = (\quad)_B$

结果： $(37)_D = (100101)_B$

1.2.3 二进制与十进制之间的转换

- 凑数法整数部分转换：将目标数展开为二进制权位多项式

◆ 例：将 $(1033)_D$ 转换为二进制数

解：由于 2^{10} 为 1024，而 $1033 - 1024 = 9 = 2^3 + 2^0$ ，

所以对应二进制数 $b_{10}=1$ ， $b_3=1$ ， $b_0=1$ ，其余各系数均为 0，

所以得 $(1033)_D = (100\ 0000\ 1001)_B$

- 问题： $(33)_D = (10\ 0001)_B$ $32 + 1 = 2^5 + 2^0$

$$(1048577)_D = (1\ 0000\ 0000\ 0000\ 0000\ 0001)_B$$

$$1024 \times 1024 + 1 = 2^{20} + 2^0$$

1.2.3 二进制与十进制之间的转换

- 连乘法小数部分转换：乘 2 取整，先高后低，达标为止

◆ 将十进制小数部分乘以 2，所得整数由高到低位排列

例： $(0.6875)_D = (\quad)_B$

$$\begin{array}{r} 0.6875 \\ \times 2 \\ \hline 1.3750 \\ \times 2 \\ \hline 0.7500 \\ \times 2 \\ \hline 1.5000 \\ \times 2 \\ \hline 1.0000 \end{array}$$

结果： $(0.6875)_D = (0.1011)_B$

1.2.3 二进制与十进制之间的转换

- 例 1.2.4 $(0.706)_D$ 转换成二进制数，误差不大于 2^{-10}

解：要求最多 10 位小数

$$0.706 \times 2 = 1.412 \quad b_{-1} = 1$$

$$0.412 \times 2 = 0.824 \quad b_{-2} = 0$$

$$0.824 \times 2 = 1.648 \quad b_{-3} = 1$$

$$0.648 \times 2 = 1.296 \quad b_{-4} = 1$$

$$0.296 \times 2 = 0.592 \quad b_{-5} = 0$$

$$0.592 \times 2 = 1.184 \quad b_{-6} = 1$$

$$0.184 \times 2 = 0.368 \quad b_{-7} = 0$$

$$0.368 \times 2 = 0.736 \quad b_{-8} = 0$$

$$0.736 \times 2 = 1.472 \quad b_{-9} = 1$$

$$0.472 \times 2 = 0.944 \quad b_{-10} = 1$$

(四舍五入)

所以： $(0.706)_D = (0.1011010011)_B$

1.2.4 十六进制和八进制

- 十六 / 八进制 (下标分别是 **H** / **O** , 或 **16** / **8**)

- ◆ 位模分别为 **16** / **8**
- ◆ 数字符号分别有 **16** / **8** 个, 分别是 $\{0 \sim 9, A \sim F\} / \{0 \sim 7\}$
- ◆ 逢十六 / 八进一, 即 $1_H + F_H = 10_H / 1_O + 7_O = 10_O$
- ◆ 第 k 位的位权分别为 $16^k / 8^k$
- ◆ 按其位权展成多项式的形式

$$\begin{aligned} (\text{BD2.3C})_H &= B \times 16^2 + D \times 16^1 + 2 \times 16^0 + 3 \times 16^{-1} + C \times 16^{-2} \\ &= 11 \times 16^2 + 13 \times 16^1 + 2 \times 16^0 + 3 \times 16^{-1} + 12 \times 16^{-2} \end{aligned}$$

$$(\text{752.34})_O = 7 \times 8^2 + 5 \times 8^1 + 2 \times 8^0 + 3 \times 8^{-1} + 4 \times 8^{-2}$$

- ◆ 可用 1 位十六 / 八进制数分别表示 **4** / **3** 位二进制数

1.2.4 十六进制和八进制

● 二进制数转十六 / 八进制方法

- ◆ 从小数点开始，将整数和小数部分每四 / 三位一组，不足四 / 三位的分别在整数左边和小数右边补 0，并用等值的十六 / 八进制符号替代各组二进制符号

- ◆ 例：(10111001011.01001)_B = ()_H

0101 1100 1011 . 0100 1000

5 C B . 4 8

(10111001011.01001)_B = ()_O

010 111 001 011 . 010 010

2 7 1 3 . 2 2

1.2.4 十六进制和八进制

● 十六 / 八进制转二进制方法

◆ 用四 / 三位二进制符号替代一位十六 / 八进制符号，小数位置不变

◆ 例 1.2.7 : $(F15.6)_H = (\quad)_B$

F 1 5 . 6

1111 0001 0101 . 0110

$(36.24)_O = (\quad)_B$

3 6 . 2 4

011 110 . 010 100

1.2.5 互补数

- 设 x_R, y_R 是 N 位 R 进制数，即其数模或计数空间为 R^N
 - ◆ 若 $x_R = R^N - y_R$ ，则称 x_R 与 y_R 关于 R 互补
 - 特例：0 关于 R 的互补数是 R^N ，而 R^N 并非 N 位 R 进制数
 - ◆ 若 $x_R = R^N - 1 - y_R$ ，则称 x_R 与 y_R 关于 $R - 1$ 互补
 - 关于 $R - 1$ 的互补数第 n 位之和等于 $R - 1$ ，即 $x_{R[n]} + y_{R[n]} = R - 1$
 - ◆ 例：4 位十进制数 1234

■ 关于 10 的补是： $10^4 - 1234 = 10000 - 1234 = 8766$

$\begin{array}{r} \cdot \cdot \cdot \cdot \\ 10000 \\ - 1234 \\ \hline 8766 \end{array}$	$\begin{array}{r} 9999 \\ - 1234 \\ \hline 8765 \end{array}$
--	--

■ 关于 9 的补是： $10^4 - 1 - 1234 = 9999 - 1234 = 8765$

◆ 例：4 位二进制数 1011

■ 关于 2 的补是： $2^4 - 1011_B = 10000_B - 1011_B = 0101_B$

$\begin{array}{r} \cdot \cdot \cdot \cdot \\ 10000 \\ - 1011 \\ \hline 0101 \end{array}$	$\begin{array}{r} 1111 \\ - 1011 \\ \hline 0100 \end{array}$
--	--

■ 关于 1 的补是： $2^4 - 1 - 1011_B = 1111_B - 1011_B = 0100_B$

- 二进制数 x_B 和 y_B 关于 1 互补，即互反，表示为 $y_B = \sim x_B$
- 二进制数 x_B 和 y_B 关于 2 互补，则有 $y_B = \sim x_B + 1$

1.2.6 二进制数的表示方法

- N 位二进制码 $(b_{N-1}b_{N-2} \dots b_0)_B$ 可用于表示无符号和有符号数
- 一般用二进制自然码表示 $[0, 2^N - 1]$ 区间的无符号数 $x_D = \sum_{k=0}^{N-1} b_k 2^k$
- 有符号数用最高位（MSB）表示符号位，0 为正、1 为负
 - ◆ 用原码表示 $[0, 2^{N-1} - 1]$ 区间的正数， $N-1$ 位值位即为其绝对值
 - ◆ 负数有三种形式表示 $N-1$ 位值位
 - 原码：值位是其绝对值，数区间为 $[-(2^{N-1} - 1), -0]$
原码对应的十进制数为 $x_D = (-1)^{b_{N-1}} \times \sum_{k=0}^{N-2} b_k 2^k$
 - 反码：值位是其绝对值关于 1 的补数，数区间为 $[-(2^{N-1} - 1), -0]$
反码对应的十进制数为 $x_D = -b_{N-1}(2^{N-1} - 1) + \sum_{k=0}^{N-2} b_k 2^k$
 - 补码：值位是其绝对值关于 2 的补数，数区间为 $[-2^{N-1}, -1]$
补码对应的十进制数为 $x_D = -b_{N-1}2^{N-1} + \sum_{k=0}^{N-2} b_k 2^k$

1.2.6 二进制数的表示方法

●例：三位二进制数的码

二进制码	无符号数	有符号数		
000	0	+0		
001	1	+1		
010	2	+2		
011	3	+3		
100	4	原码表示	反码表示	补码表示
		-0	-3	-4
101	5	-1	-2	-3
110	6	-2	-1	-2
111	7	-3	-0	-1
展开式	$\sum_{k=0}^{N-1} b_k 2^k$	$(-1)^{b_{N-1}} \times \sum_{k=0}^{N-2} b_k 2^k$	$-b_{N-1}(2^{N-1}-1) + \sum_{k=0}^{N-2} b_k 2^k$	$-b_{N-1}2^{N-1} + \sum_{k=0}^{N-2} b_k 2^k$

1.3 有符号二进制的加法运算

- 为简化电路，数字系统采用加法器实现减运算
- “ $-x_B$ ” 转义为求 x_B 关于 2 的补数
 - ◆ $-x_B = \sim x_B + 1_B$
 - ◆ $y_B - x_B = y_B + (\sim x_B) + 1_B$
 - ◆ 例如 4 位二进制有符号数 $y_B = 0111_B$ ， $x_B = 0011_B$ ，求 $y_B - x_B$

$$\begin{aligned} y_B - x_B &= y_B + (-x_B) = y_B + (\sim x_B + 1_B) \\ &= 0111_B + (1100_B + 1_B) = 0111_B + 1101_B \\ &= 0100_B \end{aligned}$$

0111
+ 1101
<u>1111</u>
0100

- 快速求补：符号为 1 的最低位及更低位不变，其他位取反
 - ◆ 如 0110 关于 2 的补为 1010 ， 0111 关于 2 的补为 1001
- 二进制数关于 2 的补数的补数，就是其本身

1.3 有符号二进制的加法运算

● 有符号数的溢出

◆ 例 1.3.8 四位二进制有符号数运算 $5 + 7 = ?$

解: $5 + 7$

$$= 0101_B + 0111_B$$

$$= 1100_B$$

$$\begin{array}{r} 0101 \\ + 0111 \\ \hline 0111 \\ 1100 \end{array}$$

- ◆ 结果 1100，是 -4 的补码形式，而正确结果应为 +12
- ◆ 此例只有 3 位数值位，能表示的最大数为 +7，故溢出导致出错
- ◆ 解决溢出的办法：位扩展

1.3 有符号二进制的加法运算

● 有符号数溢出的判别

- ◆ 符号相异的两数之和不会产生溢出
- ◆ 如何判断是否产生溢出？

$$\begin{array}{r}
 + 4 \\
 + 3 \\
 \hline
 + 7
 \end{array}
 \quad
 \begin{array}{r}
 0100 \\
 + 0011 \\
 \hline
 0000 \\
 0111
 \end{array}$$

$$\begin{array}{r}
 + 2 \\
 + 6 \\
 \hline
 + 8
 \end{array}
 \quad
 \begin{array}{r}
 0010 \\
 + 0110 \\
 \hline
 0110 \\
 1000 \text{ X}
 \end{array}$$

$$\begin{array}{r}
 - 5 \\
 + - 3 \\
 \hline
 - 8
 \end{array}$$

$$\begin{array}{r}
 - 3 \\
 + - 6 \\
 \hline
 - 9
 \end{array}$$

$$\begin{array}{r}
 5 \\
 + - 2 \\
 \hline
 3
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 + 1110 \\
 \hline
 1100 \\
 0011
 \end{array}$$

次高进位

最高进位

符号位

$$-2^{N-1} + \sum_{k=0}^{N-2} b_k 2^k$$

值位3位0是8的补数，故结果为-8

$$\begin{array}{r}
 1011 \\
 + 1101 \\
 \hline
 1111 \\
 1000 \\
 1101 \\
 + 1010 \\
 \hline
 1000 \\
 0111 \text{ X}
 \end{array}$$

- ◆ 方法1：同符号两数之和的符号若异于最高进位，则溢出
- ◆ 方法2：同符号两数之和的符号若异于源操作数符号，则溢出
- ◆ 方法3：有符号两数之和的最高进位若异于次高进位，则溢出

- 编码：按特定码制生成各种信息代码的过程
 - 数字系统的信息分为两类：
 - ◆ 数，信息符号（如文字符、控制符、事物标识符等）
 - 数的二进制代码
 - ◆ 无符号数的二进制码：一般指自然码，即二进制数的自然表示
 - 格雷码是另一种常用的二进制数表示，其具有循环码特征
 - ◆ 有符号数：原码表示的正数，补码表示的负数
 - 信息符号的二进制代码
 - ◆ 是信息符号的二进制标识， n 位二进制码可标识 2^n 个信息符号
 - ◆ 如：BCD 码表示阿拉伯数字符号，ASCII 码表示文本、控制符
- 问题：每个阿拉伯数字符号至少需要几位二进制代码表示？

1.4.1 格雷码

- 格雷码的特点：
 - ◆ 两组相邻码间只有一位不同
 - ◆ 首尾码即 0000_{GRAY} 和 1000_{GRAY} 也只有一位不同，故为典型循环码
- 主要优点：
 - ◆ 防止代码变化过程中产生短暂错误
- 例：4 位计数器
 - ◆ 普通码，+1 或 -1 时可能引起若干位变化，因信号传递路径速度不一致，会出现短暂的错误码

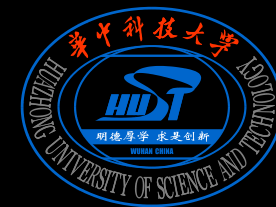
+1

0	0	0	0	1
1	1	0	0	0
1	1	1	0	0
1	0	0	0	0

四位二进制自然码与 Gray 码

十进制数	二进制码	Gray 码
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

1.4.2 BCD 码 (Binary Code Decimal)



- BCD 码： 10 个十进制数字符号的 4 位二进制代码
- 4 位二进制码可表示 16 个符号，选其中 10 个作为 BCD 码
- 常用的 BCD 码：
 - ◆ 有权码： 8421 码， 2421 码， 5421 码
 - ◆ 无权码： 余 3 码， 余 3 循环码

1.4.2 BCD 码 (Binary Code Decimal)



- 8421 码：有权码，该码由高到低位，按 8、4、2、1 位权展开多项式之和等于对应十进制符号所表示的数

$5 = 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$ ，故 '5' 的 8421BCD 码为 0101_{8421BCD}

'25.8' 的 8421 码为 (0010 0101.1000)_{8421BCD}

(0001 0010 0000 1000)_{8421BCD} 表示 '1208'

- 特点：直观

◆ 不包含后 6 个二进制符号组合：

$1010_B, 1011_B, 1100_B, 1101_B, 1110_B, 1111_B$



1.4.2 BCD 码 (Binary Code Decimal)

- 2421 码：有权码，该码由高到低位，按 2、4、2、1 位权展开多项式之和等于对应十进制符号所表示的数，且 '0' ~ '4' 的 2421 码最高位是 0，'5' ~ '9' 的 2421 码最高位是 1。

$5 = 1 \times 2 + 0 \times 4 + 1 \times 2 + 1 \times 1$ ，故 '5' 的 2421BCD 码为 $1011_{2421BCD}$

'258' 的 2421 码为 $(0010 \ 1011 \ 1110)_{2421BCD}$

$(0010 \ 0001 \ 1110 \ 1011)_{2421BCD}$ 是 '2185' 的码

- 特点：

- ◆ 对 9 自补，0 和 9、1 和 8、2 和 7、3 和 6、4 和 5 的码，各位互反

- ◆ 不包含中间 6 个二进制符号组合：

$0101_B, 0110_B, 0111_B, 1000_B, 1001_B, 1010_B$

$0000_{2421BCD} - 1111_{2421BCD}$
 $0001_{2421BCD} - 1110_{2421BCD}$
 $0010_{2421BCD} - 1101_{2421BCD}$
 $0011_{2421BCD} - 1100_{2421BCD}$
 $0100_{2421BCD} - 1011_{2421BCD}$

1.4.2 BCD 码 (Binary Code Decimal)

● 余 3 码 (Excess-3) , 无权码

- ◆ 由十进制符号的 8421 码 + 0011_B 得到对应的余 3 码

例如: '258' 的码为 $(0101\ 1000\ 1011)_{\text{EXCESS3BCD}}$

$(1000\ 1001\ 1001\ 1011)_{\text{EXCESS3BCD}}$ 是 '5668' 的码

● 特点:

$0011_{\text{EXCESS3BCD}} - 1100_{\text{EXCESS3BCD}}$
 $0100_{\text{EXCESS3BCD}} - 1011_{\text{EXCESS3BCD}}$
 $0101_{\text{EXCESS3BCD}} - 1010_{\text{EXCESS3BCD}}$
 $0110_{\text{EXCESS3BCD}} - 1001_{\text{EXCESS3BCD}}$
 $0111_{\text{EXCESS3BCD}} - 1000_{\text{EXCESS3BCD}}$

- ◆ 对 9 自补

- ◆ 与对应十进制符号运算的进位保持一致

5	1 0 0 0	$_{\text{EXCESS3BCD}}$		
+	6	+	1 0 0 1	$_{\text{EXCESS3BCD}}$
1 1	1 0 0 0	1	$_{\text{EXCESS3BCD}}$	

- ◆ 不包含前后两端各 3 个二进制符号组合:

$0000_B, 0001_B, 0010_B, 1101_B, 1110_B, 1111_B$

1.4.2 BCD 码 (Binary Code Decimal)

● 表 1.4.1 几种常用的 BCD 代码

十进制数	8421 码	2421 码	5421 码	余 3 码	余 3 循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

无末尾 6 个
四位自然码

无中间 6 个
四位自然码

无中、尾各 3 个
四位自然码

无头、尾各 3 个
四位自然码

无头、尾各 3 个
四位格雷码

- 字符代码：对字母和符号编制的代码
- ASCII 码：美国信息交换标准代码
 - ◆ American Standard Code for Information Interchange
 - ◆ 共 128 个代码，可表示英文字母、阿拉伯数字、标点符、运算符、控制符等，普遍用于键盘指令和文本数据
- ASCII 码应该是几位码？
 - 7 位，列码 3 位 $b_6b_5b_4$ ，行码 4 位 $b_3b_2b_1b_0$

1.4.3 ASCII 码

- 如： 'K' 的列码是 100_B，行码是 1011_B，即 (100 1011)_{ASCII}
 $b_6b_5b_4$ (column)

$b_3b_2b_1b_0$	Row (hex)	000 0	001 1	010 2	011 3	100 4	101 5	110 6	111 7
0000	0	NUL	DLE	SP	0	@	P	'	p
0001	1	SOH	DC1	!	1	A	Q	a	q
0010	2	STX	DC2	"	2	B	R	b	r
0011	3	ETX	DC3	#	3	C	S	c	s
0100	4	EOT	DC4	\$	4	D	T	d	t
0101	5	ENQ	NAK	%	5	E	U	e	u
0110	6	ACK	SYN	&	6	F	V	f	v
0111	7	BEL	ETB	'	7	G	W	g	w
1000	8	BS	CAN	(	8	H	X	h	x
1001	9	HT	EM	)	9	I	Y	i	y
1010	A	LF	SUB	*	:	J	Z	j	z
1011	B	VT	ESC	+	;	K	[	k	{
1100	C	FF	FS	,	<	L	\	l	
1101	D	CR	GS	-	=	M	]	m	}
1110	E	SO	RS	.	>	N	^	n	~
1111	F	SI	US	/	?	O	_	o	DEL

● 逻辑运算

- ◆ 0 和 1 表示逻辑状态时，各逻辑状态按特定因果关系进行的运算
- ◆ 逻辑运算的数学工具是逻辑代数

● 逻辑代数与普通代数

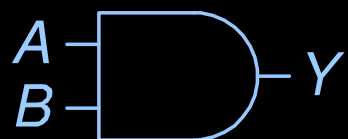
- ◆ 相同点：由字母表示的逻辑变量和逻辑运算符组成
- ◆ 不同点：逻辑代数中的变量只有 0 和 1 两个值
- ◆ 在逻辑代数中，有与、或、非三种基本的逻辑运算
- ◆ 逻辑运算的描述方式：逻辑代数表达式、真值表、逻辑图、卡诺图、波形图和硬件描述语言 (HDL) 等

1.5 二值逻辑变量与基本逻辑运算

● 与、或、非、时延缓冲

◆ ANSI / IEEE 通用（异形）符号： EDA 通用

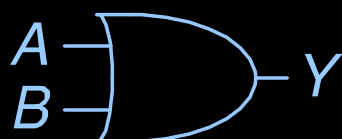
AND



$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

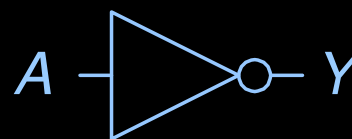
OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

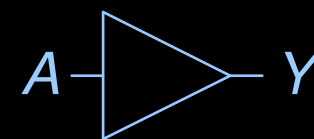
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

BUF

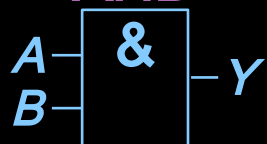


$$Y = A$$

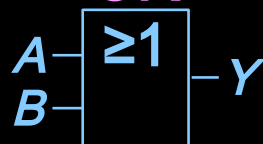
A	Y
0	0
1	1

◆ ISO / IEC 标准符号： GB 采纳

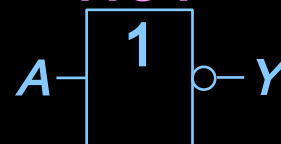
AND



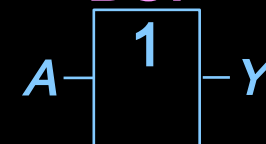
OR



NOT



BUF

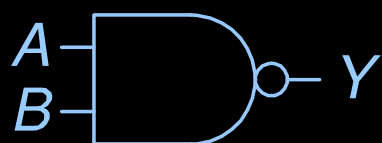


1.5 二值逻辑变量与基本逻辑运算

● 与非、或非、异或、同或（异或非）

◆ ANSI / IEEE 通用（异形）符号：EDA 通用

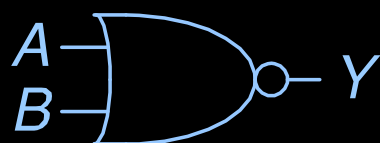
NAND



$$Y = \overline{A \cdot B}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

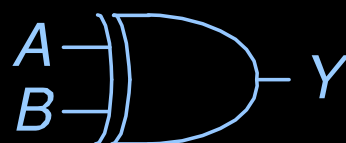
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

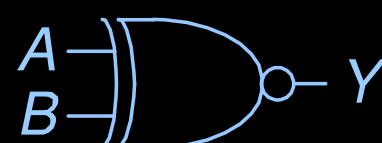
XOR



$$Y = A \oplus B \\ = \overline{A} \cdot B + A \cdot \overline{B}$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

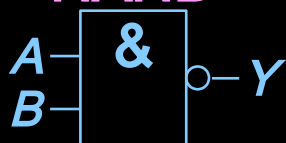


$$Y = \overline{A \oplus B} = A \odot B \\ = \overline{A} \cdot \overline{B} + A \cdot B$$

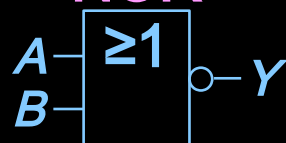
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

◆ ISO / IEC 标准符号：GB 采纳

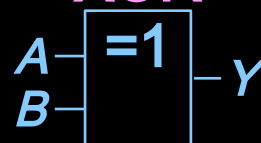
NAND



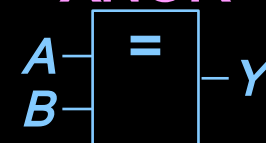
NOR



XOR



XNOR



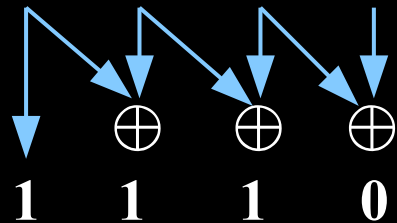
1.5 二值逻辑变量与基本逻辑运算

● 二进制码与格雷码互相转换的方法

二进制码: $B = b_{n-1}b_{n-2}\dots b_1b_0$ 格雷码: $G = g_{n-1}g_{n-2}\dots g_1g_0$

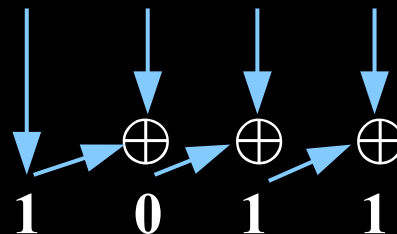
● Binary $\rightarrow$ Gray 方法: $g_{n-1} = b_{n-1}$ $g_i = b_{i+1} \oplus b_i$

例如: $(1\ 0\ 1\ 1)_B = (1110)_{GRAY}$



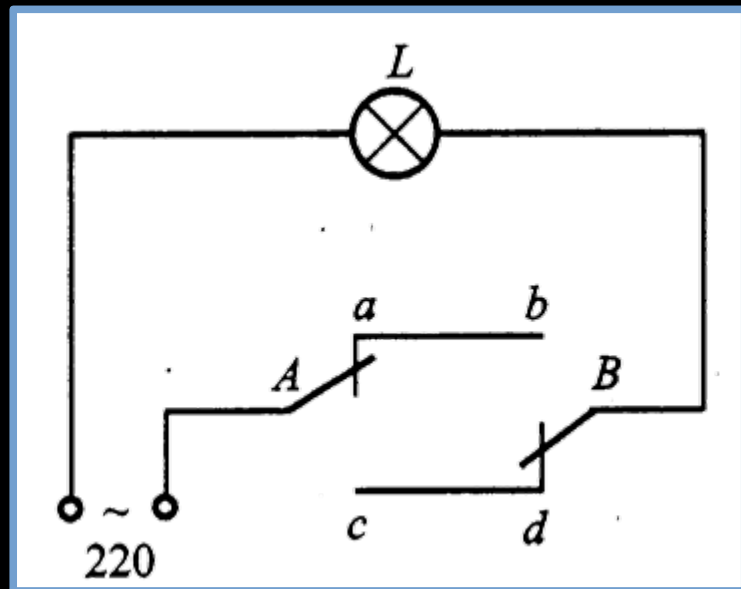
● Gray $\rightarrow$ Binary 方法: $b_{n-1} = g_{n-1}$ $b_i = b_{i+1} \oplus g_i$

例如: $(1\ 1\ 1\ 0)_{GRAY} = (1011)_B$



1.6 逻辑函数的建立及其表示方法

- 可用真值表、逻辑函数表达式、逻辑图、波形图和卡诺图描述输入和输出的逻辑关系
 - ◆ 如图控制楼梯照明灯的电路，开关 A 、 B 分别在楼下和楼下，设开关 A 、 B 同时上扳或同时下扳，灯才亮，试用逻辑表达式描述
 - ◆ 设 L 表示灯状态，亮为 1；灭为 0
 - ◆ A 和 B 表示开关，上扳为 1，下扳 0
 - ◆ A 、 B 都为上扳，即 $A = B = 1$ 时，或都为下扳，即 $A = B = 0$ 时，灯才亮，即 $L = 1$ 。逻辑关系？



1.6 逻辑函数的建立及其表示方法

● 描述上述同或逻辑的 4 种方法

● 真值表

◆ 真值表是输入变量所有取值组合与函数值间的对应关系列成表格

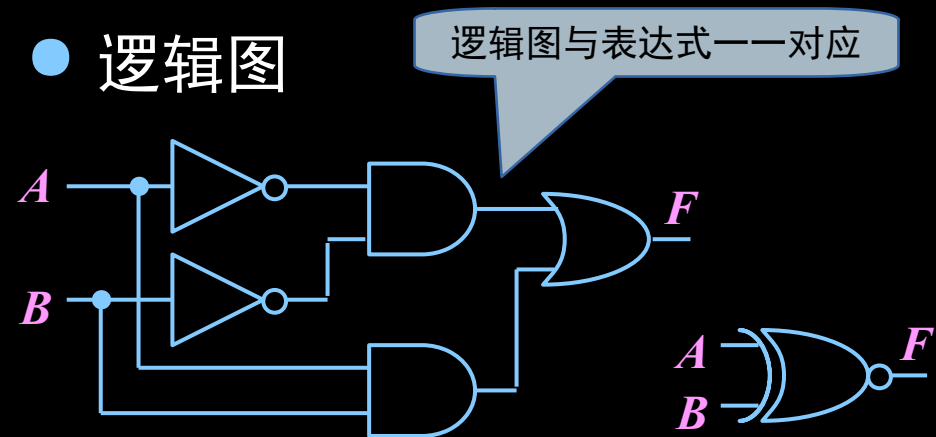
A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

● 逻辑表达式

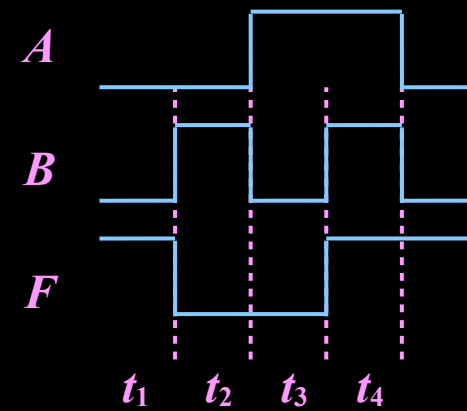
$$F = \overline{A \oplus B} = A \odot B$$

$$= \overline{A} \overline{B} + A B$$

● 逻辑图



● 波形图



康华光教材 7 版

1.1.2

1.2.3 (1, 3)

1.2.8

1.4.1 (2, 4)

1.4.3 (1, 3)

1.6.1